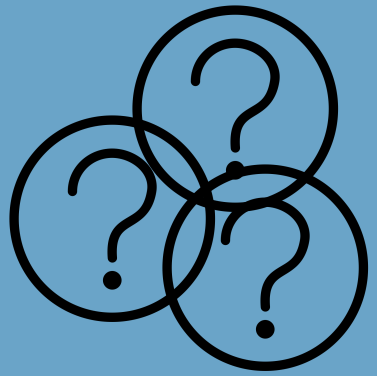
Information Retrieval

NLP II 2026

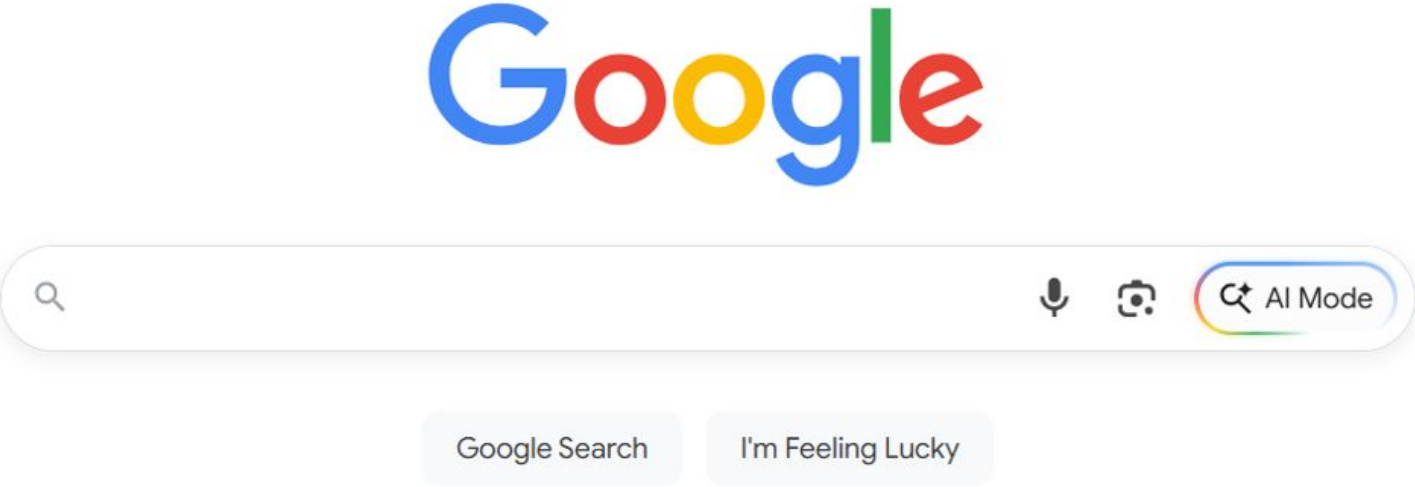
Assoc. Prof. Attapol Thamrongrattanarit

What is information need?
Where do we get such information?



Information Retrieval

- The user has...
  - A need for information
  - Some collection of documents
- The user wants to find a **relevant** document, that is...
  - A document (or documents)
  - In a collection
  - That satisfy/satisfies their need



Web Search and Retrieval-Augmented Generation

Retrieval-Augmented **Generation**

Google

จุฬา TCAS รอบ 3 คณะอักษร สอบฝรั่งเศส รับกี่คน

AI Mode | All | Images | Videos | Shopping | News | Short videos | More | Tools

AI Overview

จุฬาลงกรณ์มหาวิทยาลัย คณะอักษรศาสตร์ รอบ 3 Admission (TCAS69) เปิดรับนิสิตร่วมกันในวิชาภาษาที่ 3 (รวมฝรั่งเศส, จีน, ญี่ปุ่น, เกาหลี, เยอรมัน, สเปน, บาลี) จำนวนทั้งหมด **187 คน** โดยใช้คะแนน A-Level ภาษาฝรั่งเศส, ภาษาไทย, สังคม และภาษาอังกฤษ เป็นเกณฑ์หลักในการคัดเลือก Facebook

สรุปรายละเอียดที่สำคัญ

- **โครงการ/รอบ:** รอบ 3 Admission
- **วิชาที่ใช้:** A-Level (ภาษาฝรั่งเศส 25%, ภาษาไทย 25%, สังคม 25%, อังกฤษ 25%)
- **คุณสมบัติ:** GPAX 2.75
- **จำนวนรับ:** รับร่วมกับภาษาอื่นๆ รวม 187 คน Facebook

ทั้งนี้ จำนวนรับจริงอาจมีการเปลี่ยนแปลงได้ โปรดตรวจสอบรายละเอียดประกาศอย่างเป็นทางการจาก คณะอักษรศาสตร์ จุฬาลงกรณ์มหาวิทยาลัย และเว็บไซต์ mytcas.com อีกครั้ง Facebook +1

Dive deeper in AI Mode

สรุปเกณฑ์ #อักษรจุฬา รอบ 3 Admission #TCAS69 เลือกสอบวิชาภาษาที่ 3 รับ ...

Oct 19, 2568 BE — 📌สรุปเกณฑ์ #อักษรจุฬา รอบ 3 Admission #TCAS69 🚩เลือกสอบวิชาภาษาที่ 3 รั...

Facebook

จุฬาประกาศเกณฑ์รับรอบสามปีแล้วค่ะ - Facebook

Oct 31, 2568 BE — จุฬาฯ เคาะเกือบทุกคณะใช้ TGAT ในรอบ 3 ADMISSION ปีนี้ #TCAS69 . ✅...

Facebook

wongnai

agoda

What if we can ask any type of questions ?

Shopee

LINE

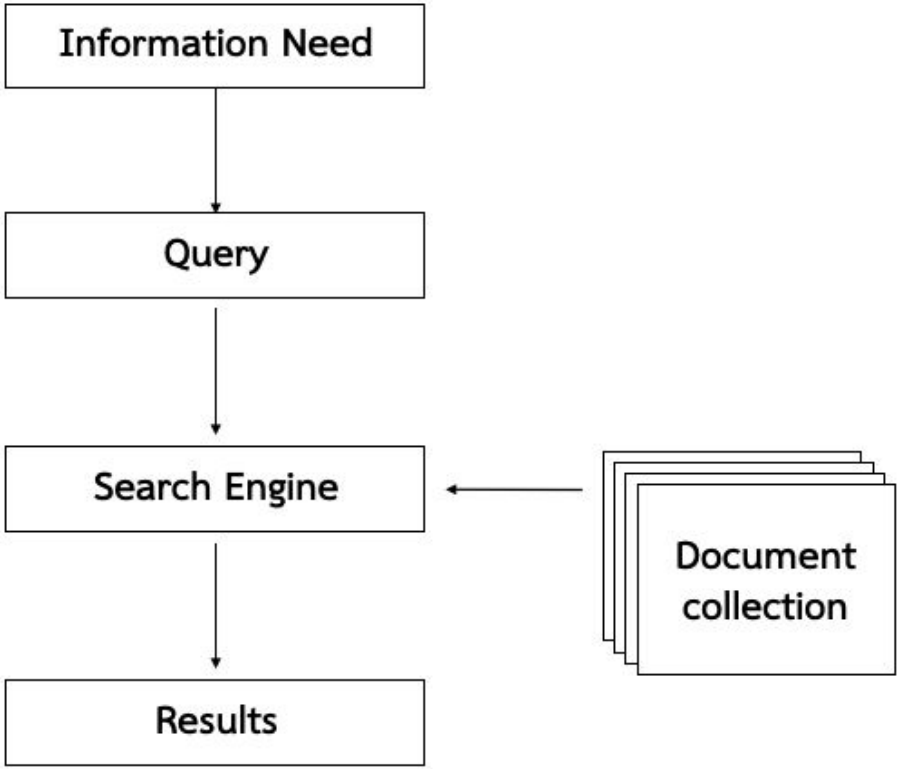
Information Retrieval

- Information Retrieval (search) = การค้นคืน document จาก document collection ที่ตอบโจทย์ความต้องการทางข้อมูล (Information need)
- Retrieval-Augmented Generation (search + LLM) = การใช้เนื้อหาที่ค้นคืนมาจาก document collection มาสรรค์สร้างคำตอบที่ตอบโจทย์ความต้องการของข้อมูล

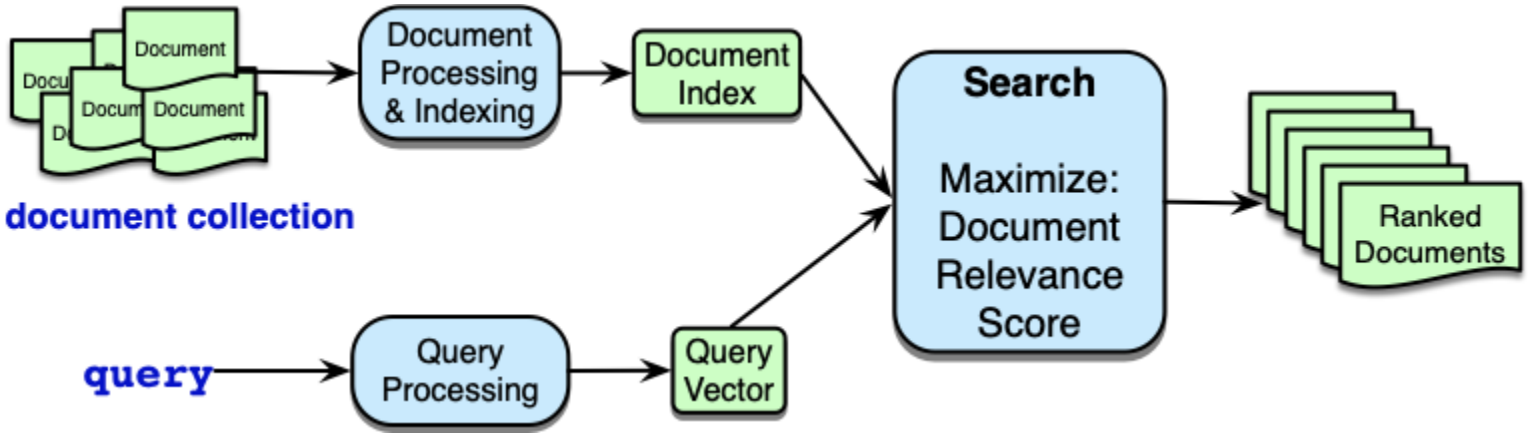
Classic Search Model

Information need and document collection differ based on the tasks.

Results are ranked documents



Ad-hoc Retrieval



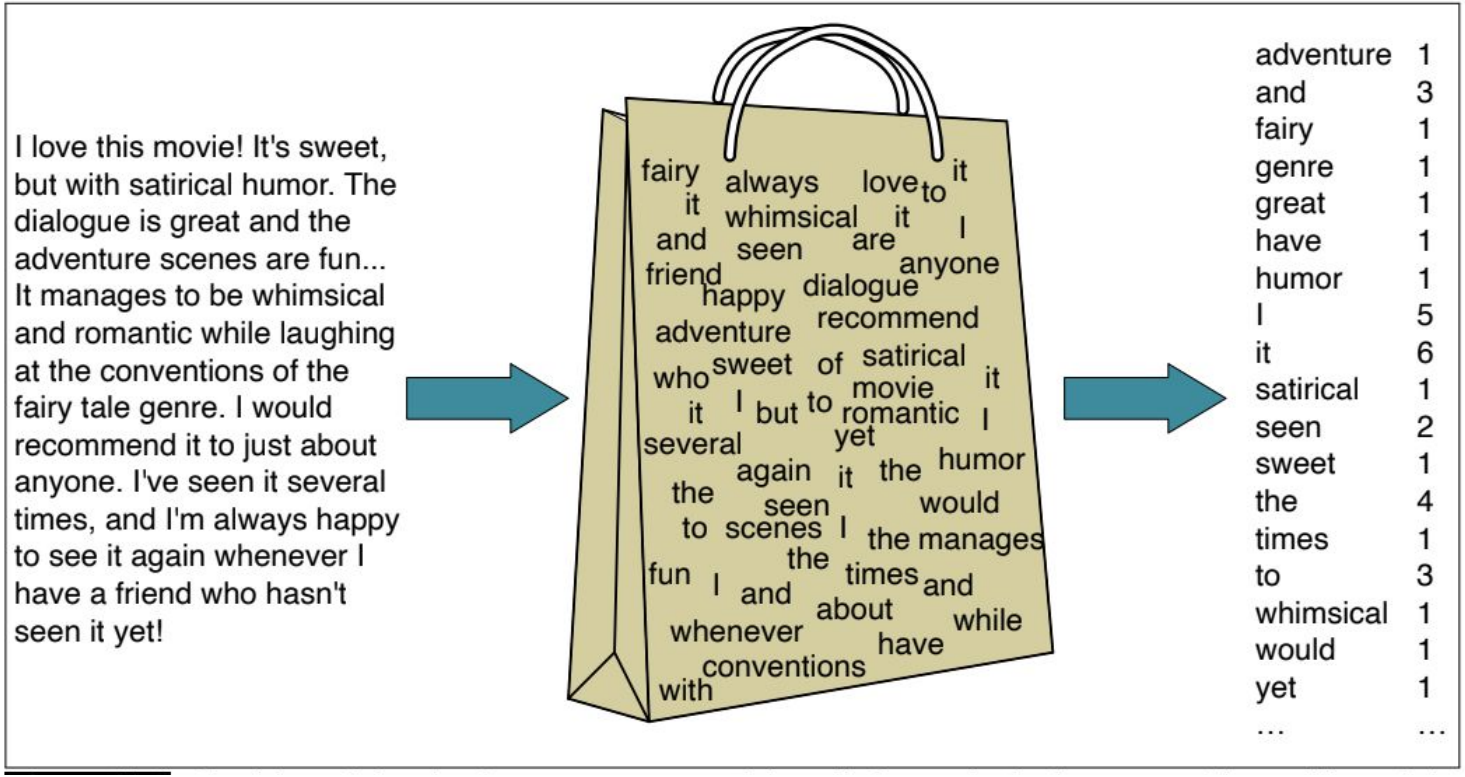
Document Relevance Score

- The goal is to assign a score to each document according to whether it meets the user's information need (indicated by the query)
  - We approximate this by the textual similarity between the query and the document, both of which is a vector/embedding.

Two Architectures

- **Sparse retrieval** represents query and doc as vectors of word counts (like bag-of-word logistic regression) weighted by tf-idf or others
- **Dense retrieval** represents query and doc as embeddings (like BERT model)
- In both cases, similarity is defined as the *dot product* or *cosine* between query and document representations

Sparse Retrieval: The Vector Model of IR



Bag-of-words Model

Vector Representation of Document

adventure	1
and	3
fairy	1
genre	1
great	1
have	1
humor	1
I	5
it	6
satirical	1
seen	2
sweet	1
the	4
times	1
to	3
whimsical	1
would	1
yet	1
…	…

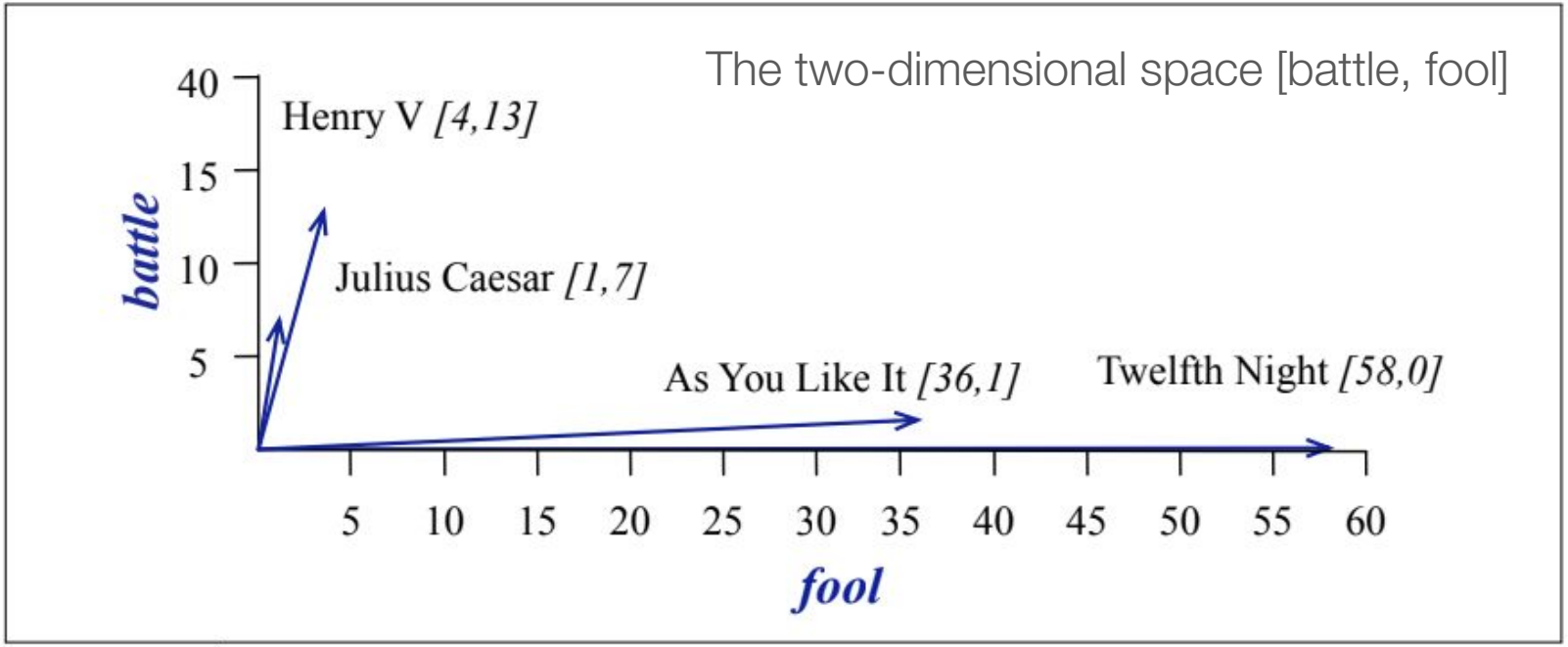
[1 3 1 1 1 1 1 5 6 1 2 1 4 1 3 1 1 1]

Term-Document (TD) Matrix

- Each document is represented by a vector of words (in IR, we call 'terms' because what we count might not linguistic 'words')

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

Visualizing Document Vectors



Term-Document (TD) Matrix

- Comedies have similar vectors, which are different from histories
  - Comedies have more *fools* and *wit* and fewer *battles*

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

Vector Representations of Queries and Documents

- Suppose we are looking for a witty, foolish play
  - Query = `"fool wit"`

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

Query
0
0
1
1

Choose the most similar document to the query

- Which of $\mathbf{d}_1$, $\mathbf{d}_2$, $\mathbf{d}_3$, $\mathbf{d}_4$ is most similar to $\mathbf{q}$?

	$\mathbf{d}_1$	$\mathbf{d}_2$	$\mathbf{d}_3$	$\mathbf{d}_4$	$\mathbf{q}$
	As You Like It	Twelfth Night	Julius Caesar	Henry V	Query
battle	1	0	7	13	0
good	114	80	62	89	0
fool	36	58	1	4	1
wit	20	15	2	3	1

Similarity methods are variants of dot product

- The dot product is **q · d**
  - **score (q, d1)** = **q · d1** = …

	d_1	d_2	d_3	d_4	q
	As You Like It	Twelfth Night	Julius Caesar	Henry V	Query
battle	1	0	7	13	0
good	114	80	62	89	0
fool	36	58	1	4	1
wit	20	15	2	3	1

In fact, we use cosine

$$\text{score}(q,d) = \cos(\mathbf{q},\mathbf{d}) = \frac{\mathbf{q}\cdot\mathbf{d}}{|\mathbf{q}||\mathbf{d}|}$$

$$= \frac{\mathbf{q}}{|\mathbf{q}|}\cdot\frac{\mathbf{d}}{|\mathbf{d}|}$$

Let's try this

$$\text{score}(q,d) = \cos(\mathbf{q},\mathbf{d}) = \frac{\mathbf{q}\cdot\mathbf{d}}{|\mathbf{q}||\mathbf{d}|} = \frac{\mathbf{q}}{|\mathbf{q}|}\cdot\frac{\mathbf{d}}{|\mathbf{d}|}$$

	d_1	d_2	d_3	d_4	q
	As You Like It	Twelfth Night	Julius Caesar	Henry V	Query
battle	1	0	7	13	0
good	114	80	62	89	0
fool	36	58	1	4	1
wit	20	15	2	3	1

$$\|q\| = \sqrt{0^2 + 0^2 + 1^2 + 1^2} = \sqrt{2} = 1.414$$

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

Query
0
0
1
1

For d_1

$$\text{score}(q,d) = \cos(\mathbf{q},\mathbf{d}) = \frac{\mathbf{q}\cdot\mathbf{d}}{|\mathbf{q}||\mathbf{d}|} = \frac{\mathbf{q}}{|\mathbf{q}|}\cdot\frac{\mathbf{d}}{|\mathbf{d}|}$$

Dot product:

$$q \cdot d_1 = 1 \cdot 36 + 1 \cdot 20 = 56$$

Norm:

$$\|d_1\| = \sqrt{1^2 + 114^2 + 36^2 + 20^2} = \sqrt{14693} = 121.22$$

Cosine score:

$$\text{score}(q, d_1) = \frac{56}{1.414 \times 121.22} = 0.327$$

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

Query
0
0
1
1

For d_2

$$\text{score}(q,d) = \cos(\mathbf{q},\mathbf{d}) = \frac{\mathbf{q}\cdot\mathbf{d}}{|\mathbf{q}||\mathbf{d}|} = \frac{\mathbf{q}}{|\mathbf{q}|}\cdot\frac{\mathbf{d}}{|\mathbf{d}|}$$

Dot product:

$$q \cdot d_2 = 58 + 15 = 73$$

Norm:

$$\|d_2\| = \sqrt{0^2 + 80^2 + 58^2 + 15^2} = \sqrt{9989} = 99.94$$

Cosine score:

$$\text{score}(q, d_2) = \frac{73}{1.414 \times 99.94} = 0.516$$

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

Document	Dot product	Norm	Cosine score
d1 – As You Like It	56	121.22	0.327
d2 – Twelfth Night	73	99.94	0.516
d3 – Julius Caesar	3	62.43	0.034
d4 – Henry V	7	90.08	0.055

Query
0
0
1
1

how well q and d match?

to be fair to smaller documents

Term weighting with TF-IDF

But raw frequency is a bad representation

- Raw frequency also does not account for very common words that appear in almost every document, so these words provide little information for distinguishing documents. For example, *it the for in* appear in all documents.
- If doc 1 contains 'fool' 3 times and doc 2 contains 30 times, doc 2 is not necessarily 10 times more relevant to the query.

Common Solution for Term Weighting

- **TF-IDF**
  - Words like *the* or *it* have very low IDF

$$w_{t,d} = \mathrm{tf}_{t,d} \times \mathrm{idf}_t$$

Term frequency (TF) in the TF-IDF Algorithm

- We could imagine using raw counts

$$\mathrm{tf}_{t,d} = \mathrm{count}(t,d)$$

- But instead of using raw counts, we usually squash them a bit

$$\mathrm{tf}_{t,d} = \begin{cases} 1 + \log_{10} \mathrm{count}(t,d) & \text{if } \mathrm{count}(t,d) > 0 \\ 0 & \text{otherwise} \end{cases}$$

TF = 1 + log10 or 0

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

term	**As You Like It**	**Twelfth Night**	**Julius Caesar**	**Henry V**
battle	1.000	0.000	1.845	2.114
good	3.057	2.903	2.792	2.949
fool	2.556	2.763	1.000	1.602
wit	2.301	2.176	1.301	1.477

Document Frequency (DF)

- df_t is the number of documents a given term *t* occurs in
- For example, *Romeo* is very distinctive, in that it occurs in only one Shakespeare play

	Collection Frequency	Document Frequency
Romeo	113	1
action	113	31

Inverse Document Frequency (IDF)

$$\text{idf}_t = \log_{10}\left(\frac{N}{\text{df}_t}\right)$$

N is the total number of documents in the collection

Word	df	idf
Romeo	1	1.57
salad	2	1.27
Falstaff	4	0.967
forest	12	0.489
battle	21	0.246
wit	34	0.037
fool	36	0.012
good	37	0
sweet	37	0

Final TF-IDF weighted value for a given word

$$w_{t,d} = \mathrm{tf}_{t,d} \times \mathrm{idf}_t$$

Raw Counts

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

TF-IDF

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	0.246	0	0.454	0.520
good	0	0	0	0
fool	0.030	0.033	0.0012	0.0019
wit	0.085	0.081	0.048	0.054

TF-IDF: A Worked Example

Cosine

$$\text{score}(q,d) = \cos(\mathbf{q},\mathbf{d}) = \frac{\mathbf{q}\cdot\mathbf{d}}{|\mathbf{q}||\mathbf{d}|}$$

$$= \frac{\mathbf{q}}{|\mathbf{q}|}\cdot\frac{\mathbf{d}}{|\mathbf{d}|}$$

TF-IDF Weighted Cosine

$$\text{score}(q,d) = \sum_{t \in \mathbf{q}} \frac{\text{tf-idf}(t,q)}{\sqrt{\sum_{q_i \in q} \text{tf-idf}^2(q_i,q)}} \cdot \frac{\text{tf-idf}(t,d)}{\sqrt{\sum_{d_i \in d} \text{tf-idf}^2(d_i,d)}}$$

TF-IDF Weighted Cosine

$$\mathrm{tf}_{t,d} = \begin{cases} 1 + \log_{10} \mathrm{count}(t,d) & \text{if } \mathrm{count}(t,d) > 0 \\ 0 & \text{otherwise} \end{cases}$$

$$\mathrm{idf}_t = \log_{10}\left(\frac{N}{\mathrm{df}_t}\right)$$

$$\mathrm{score}(q,d) = \sum_{t \in \mathbf{q}} \frac{\text{tf-idf}(t,q)}{\sqrt{\sum_{q_i \in q} \text{tf-idf}^2(q_i,q)}} \cdot \frac{\text{tf-idf}(t,d)}{\sqrt{\sum_{d_i \in d} \text{tf-idf}^2(d_i,d)}}$$

TF-IDF Nano-Example

Query:	sweet love
Doc 1:	Sweet sweet nurse! Love?
Doc 2:	Sweet sorrow
Doc 3:	How sweet is love?
Doc 4:	Nurse!

$$\mathrm{tf}_{t,d} = \begin{cases} 1 + \log_{10} \mathrm{count}(t,d) & \text{if } \mathrm{count}(t,d) > 0 \\ 0 & \text{otherwise} \end{cases}$$

$$\mathrm{idf}_t = \log_{10}\left(\frac{N}{\mathrm{df}_t}\right)$$

$$\mathrm{score}(q,d) = \sum_{t \in \mathbf{q}} \frac{\text{tf-idf}(t,q)}{\sqrt{\sum_{q_i \in q} \text{tf-idf}^2(q_i,q)}} \cdot \frac{\text{tf-idf}(t,d)}{\sqrt{\sum_{d_i \in d} \text{tf-idf}^2(d_i,d)}}$$

TF-IDF Nano-Example

Query = `"sweet love"`

Word	Count	TF	DF	IDF	TF-IDF	Norm.
sweet	1					
nurse	0					
love	1					
how	0					
sorrow	0					
is	0					

TF-IDF Nano-Example

Query = `"sweet love"`

$|\mathbf{q}| = \sqrt{(0.125^2 + 0.301^2)} = \mathbf{0.325}$

Word	Count	TF	DF	IDF	TF-IDF	Norm.
sweet	1	1	3	0.125	0.125	0.383
nurse	0					
love	1	1	2	0.301	0.301	0.924
how	0					
sorrow	0					
is	0					

TF-IDF Nano-Example

Query = `"sweet love"`

Document 1 `"Sweet sweet nurse! Love?"`

$|d_1| = \sqrt{(0.163^2 + 0.301^2 + 0.301^2)} = 0.456$

Cosine = 0.137 + 0.610 = **0.747**

Word	Count	TF	TF-IDF	Norm.	× q
sweet	2	1.301	0.163	0.357	**0.137**
nurse	1	1	0.301	0.661	0
love	1	1	0.301	0.661	**0.610**
how	0				
sorrow	0				
is	0				

TF-IDF Nano-Example

Query = `"sweet love"`

Document 2 `"Sweet sorrow"`

$|d_1| = \sqrt{(0.125^2 + 0.602^2)} = 0.615$

Cosine = 0.0779

Word	Count	TF	TF-IDF	Norm.	× q
sweet	1	1	0.125	0.203	**0.0779**
nurse	0				
love	0	0	0	0	0
how	0				
sorrow	1	1	0.602	0.979	0
is	0				

Final Cosine

- Compare the scores
  - **score(q, d_1) = 0.747**
  - score(q, d_2) = 0.0779
- Analysis
  - d_1 has both terms, including 2 instances of *sweet*
  - d_2 is missing one of the terms, i.e. *love*

Search in practice

Term frequency		Document frequency		Normalization	
n (natural)	$\mathrm{tf}_{t,d}$	n (no)	1	n (none)	1
l (logarithm)	$1 + \log(\mathrm{tf}_{t,d})$	t (idf)	$\log \frac{N}{\mathrm{df}_t}$	c (cosine)	$\frac{1}{\sqrt{w_1^2+w_2^2+\ldots+w_M^2}}$
a (augmented)	$0.5 + \frac{0.5 \times \mathrm{tf}_{t,d}}{\max_t(\mathrm{tf}_{t,d})}$	p (prob idf)	$\max\{0, \log \frac{N-\mathrm{df}_t}{\mathrm{df}_t}\}$	u (pivoted unique)	$1/u$
b (boolean)	$\begin{cases} 1 & \text{if } \mathrm{tf}_{t,d} > 0 \\ 0 & \text{otherwise} \end{cases}$			b (byte size)	$1/CharLength^{\alpha}$, $\alpha < 1$
L (log ave)	$\frac{1+\log(\mathrm{tf}_{t,d})}{1+\log(\mathrm{ave}_{t \in d}(\mathrm{tf}_{t,d}))}$				

ddd.qqq
lnc.ltc

Okapi BM25

$$\text{score}(D,Q) = \sum_{i=1}^{n} \text{IDF}(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgdl}}\right)}$$

- q_i is query term i
- $f(q_i, D)$ = the frequency of term q_i in document D
- $|D|$ = the length of document D
- avgdl = average document length
- k = a positive tuning parameter that calibrates the document term frequency normalization with respect to the query term frequency normalization (typical values are between 1.2 and 2.0)
- b: a positive tuning parameter that determines the scaling factor for document length normalization (typical values are between 0.75 and 1.0)

Okapi BM25

$$\text{score}(D, Q) = \sum_{i=1}^{n} \text{IDF}(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgdl}}\right)}$$

$$\text{IDF}(q_i) = \ln\left(\frac{N - n(q_i) + 0.5}{n(q_i) + 0.5} + 1\right)$$

Efficiency: The Inverted Index

Efficiency

- The goal is to rank documents in a give collection **D** by their TF-IDF weighted cosines with query **q**
- How big is a vector?
- Do we have to consider all documents in the document collection?

Inverted Index

- How can we efficiently find all documents that contain a query term $\mathbf{q_i}$?
  - An index, which, for historical reasons, is called an **inverted index**

Inverted Index

Doc 1:	Sweet sweet nurse! Love?
Doc 2:	Sweet sorrow
Doc 3:	How sweet is love?
Doc 4:	Nurse!

Dictionary	Postings		
how	→ 3		
is	→ 3		
love	→ 1	→ 3	
nurse	→ 1	→ 4	
sorrow	→ 2		
sweet	→ 1	→ 2	→ 3

Inverted Index Creation

Term	Doc No.
how	3
is	3
love	1
love	3
nurse	1
nurse	4
sorrow	2

Term	Doc No.
sweet	2
sweet	1
sweet	1
sweet	3

1. Sort by term and document

Doc 1: Sweet sweet nurse! Love?
Doc 2: Sweet sorrow
Doc 3: How sweet is love?
Doc 4: Nurse!

2. Create linked postings list

how	→ 3		
is	→ 3		
love	→ 1	→ 3	
nurse	→ 1	→ 4	
sorrow	→ 2		
sweet	→ 1	→ 2	→ 3

Inverted Index

how	→	3		
is	→	3		
love	→	1	→ 3	
nurse	→	1	→ 4	
sorrow	→	2		
sweet	→	1	→ 2	→ 3

Inverted Index

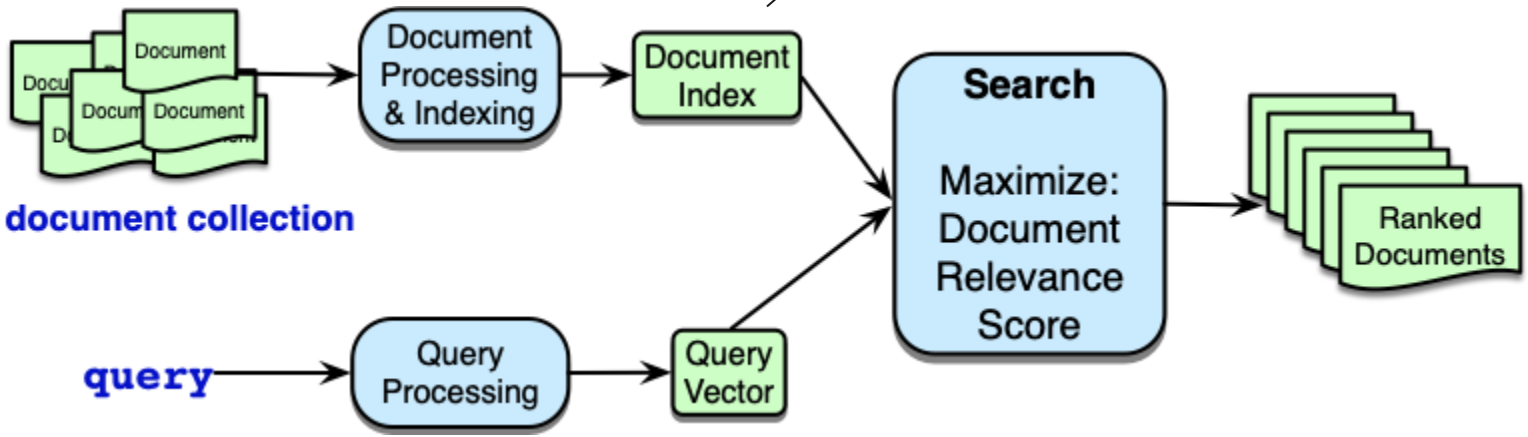
- So far this just tells us which documents to grab
- Next we need enough information to compute TF-IDF
  - For each term *t* in vocabulary:
    - The $\mathbf{df_t}$ (document frequency) of word *t*
  - For each term *t* in each document *d*:
    - The **count(t,d)** (term frequency) of word *i* in doc *d*

Inverted Index

	{DF}		Doc# [TF]
how	{1}	→	3 [1]
is	{1}	→	3 [1]
love	{2}	→	1 [1] → 3 [1]
nurse	{2}	→	1 [1] → 4 [1]
sorrow	{1}	→	2 [1]
sweet	{3}	→	1 [2] → 2 [1] → 3 [1]

Index the documents

we create this once and update when we have a new document



Evaluation of IR

Precision and Recall

		gold standard labels		
		gold positive	gold negative	
system output labels	system positive	**true positive**	**false positive**	**precision** $= \frac{tp}{tp+fp}$
	system negative	**false negative**	**true negative**	
		recall $= \frac{tp}{tp+fn}$		**accuracy** $= \frac{tp+tn}{tp+fp+tn+fn}$

- **Precision** → % of selected items that are correct
- **Recall** → % of correct items that are selected

Precision and Recall for IR

- User makes an information request
- Every document in a given collection is either:
  - **Relevant** to the user, or
  - **Not relevant** to the user
- The system then retrieves a ranked set of documents

Click data from search log

The data are already 'automatically annotated.'

query	doc id	rank	click?
1	30	1	1
1	12	2	0
1	11	3	1
1	50	4	0
2	12	1	0
2	7	2	0
2	30	3	0
2	4	4	1

Precision for IR

- **Precision** = % of retrieved documents that are relevant

$$\text{Precision} = \frac{\text{relevant retrieved documents}}{\text{all retrieved documents}}$$

Recall for IR

- **Recall** = % of relevant documents that are retrieved

$$\text{Recall} = \frac{\text{retrieved relevant documents}}{\text{all relevant documents}}$$

Compute Precision@1 2 3 4 and Recall@1 2 3 4

precision@k =

$$\frac{\text{จำนวนเอกสารที่เกี่ยวข้องที่อยู่ใน top k}}{\text{จำนวนเอกสารที่อยู่ใน top k}}$$

recall@k =

$$\frac{\text{จำนวนเอกสารที่เกี่ยวข้องที่อยู่ใน top k}}{\text{จำนวนเอกสารที่เกี่ยวข้องทั้งหมด}}$$

Assume ว่าเอกสารที่เกี่ยวข้องทั้งหมดปรากฏอยู่ใน search results ทั้งหมดแล้ว ไม่มีนอกเหนือจากนี้

query	doc id	rank	click?
1	30	1	1
1	12	2	0
1	11	3	1
1	50	4	0
2	12	1	0
2	7	2	0
2	30	3	0
2	4	4	1

Precision@k

$$P@k = \frac{\#\text{ relevant in top } k}{k}$$

Relevant documents:

$$Q_1 : \{30, 11\} \qquad Q_2 : \{4\}$$

Query 1
Ranking relevance: $(1, 0, 1, 0)$

$$P@1 = \frac{1}{1} = 1$$

$$P@2 = \frac{1}{2} = 0.5$$

$$P@3 = \frac{2}{3} \approx 0.67$$

$$P@4 = \frac{2}{4} = 0.5$$

Query 2
Ranking relevance: $(0, 0, 0, 1)$

$$P@1 = \frac{0}{1} = 0$$

$$P@2 = \frac{0}{2} = 0$$

$$P@3 = \frac{0}{3} = 0$$

$$P@4 = \frac{1}{4} = 0.25$$

query	doc id	rank	click	precision@k
1	30	1	1	1
1	12	2	0	0.5
1	11	3	1	0.67
1	50	4	0	0.5
2	12	1	0	0
2	7	2	0	0
2	30	3	0	0
2	4	4	1	0.25

Your Turn

$$\text{precision@k} = \frac{\text{จำนวนเอกสารที่เกี่ยวข้องที่อยู่ใน top k}}{\text{จำนวนเอกสารที่อยู่ใน top k}}$$

$$\text{recall@k} = \frac{\text{จำนวนเอกสารที่เกี่ยวข้องที่อยู่ใน top k}}{\text{จำนวนเอกสารที่เกี่ยวข้องทั้งหมด}}$$

Assume ว่าเอกสารที่เกี่ยวข้องทั้งหมดปรากฏอยู่ใน search results ทั้งหมดแล้ว ไม่มีนอกเหนือจากนี้

query	doc id	rank	click?
1	30	1	1
1	12	2	1
1	11	3	0
1	50	4	0
2	12	1	1
2	7	2	0
2	30	3	0
2	4	4	0

Precision@k

$$P@k = \frac{\#\text{ relevant in top } k}{k}$$

Relevant documents:

$$Q_1 : \{30, 12\} \qquad Q_2 : \{12\}$$

Query 1
Ranking relevance: $(1, 1, 0, 0)$

$$P@1 = \frac{1}{1} = 1$$

$$P@2 = \frac{2}{2} = 1$$

$$P@3 = \frac{2}{3} \approx 0.67$$

$$P@4 = \frac{2}{4} = 0.5$$

Query 2
Ranking relevance: $(1, 0, 0, 0)$

$$P@1 = \frac{1}{1} = 1$$

$$P@2 = \frac{1}{2} = 0.5$$

$$P@3 = \frac{1}{3} \approx 0.33$$

$$P@4 = \frac{1}{4} = 0.25$$

query	doc id	rank	click	precision@k
1	30	1	1	1
1	12	2	1	1
1	11	3	0	0.67
1	50	4	0	0.5
2	12	1	1	1
2	7	2	0	0.5
2	30	3	0	0.33
2	4	4	0	0.25

Precision@k & Recall@k

- We can compute F1@k using the usual formula
- The question remains which level of k we want to use to evaluate and compare two systems

Mean Average Precision

- Compute the average precision for each relevant document using the rank of that document
- **$\text{Precision}_r(d)$**
  - Find the relevant doc d
  - Find the rank r of doc d
  - Get the precision@r

Rank	Judgment	Precision_{Rank}	Recall_{Rank}
1	R	1.0	.11
2	N	.50	.11
3	R	.66	.22
4	N	.50	.22
5	R	.60	.33
6	R	.66	.44
7	N	.57	.44

Ranks 1, 3, 5, 6 but not 2 or 4

Mean Average Precision

- Descends through ranked items
- Notes precision only if item is relevant
- Calculated by taking the average of the ranked-precisions

$$\mathrm{AP} = \frac{1}{|R_r|} \sum_{d \in R_r} \mathrm{Precision}_r(d)$$

R_r is the set of relevant documents at or above r

$\mathrm{Precision}_r(d)$ is precision measured at the rank at which document *d* was found.

Mean Average Precision

- For a set of Q queries...

$$\text{MAP} = \frac{1}{|Q|} \sum_{q \in Q} \text{AP}(q)$$

query	doc id	rank	click	precision@k
1	30	1	1	1
1	12	2	0	0.5
1	11	3	1	0.67
1	50	4	0	0.5
2	12	1	0	0
2	7	2	0	0
2	30	3	0	0
2	4	4	1	0.25

Solution

Query 1
Ranking relevance: $(1, 0, 1, 0)$

$$AP(Q_1) = \frac{1}{2}\left(P@1 + P@3\right)$$

$$= \frac{1}{2}\left(1 + 0.67\right)$$

$$= 0.835 \approx 0.83$$

Query 2
Ranking relevance: $(0, 0, 0, 1)$

$$AP(Q_2) = \frac{1}{1}\left(P@4\right)$$

$$= 0.25$$

Mean Average Precision

$$MAP = \frac{AP(Q_1) + AP(Q_2)}{2}$$

$$= \frac{0.83 + 0.25}{2} = 0.54$$

IR Lesson Conclusion

01 — Information Retrieval systems find relevant documents to address information needs

02 — Use TF-IDF vectors to represent document and query and compute similiarity

03 — Evaluate ranked results with MAP